BANCO DE DADOS NÃO RELACIONAL · PROF.ª LUCINEIDE · AULA 03

Atividade Avaliativa – Modelagem no MongoDB

Flexibilidade x Estrutura — Script, análise crítica e raciocínio de arquitetura

---

PARTE 1

Banco Sem Estrutura (BD_SE)

Tarefa 1 – Inserção Livre

Três jogadores inseridos com estruturas intencionalmente diferentes para demonstrar a ausência de validação:

```
use BD_SE

db.jogadores.insertMany([
  { nome: "Felipe" },
  { nome: "Mariana", ranking: 1200, patrocinador: "TechGames" },
  { nome: "Ricardo", nick: "Shadow99", ranking: 900, redeSocial: "@ricardo_gamer"
])
```

✓ Todos os documentos inseridos com sucesso

Saída do shell:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...'),
    '2': ObjectId('...')
  }
}
```

Tarefa 2 – Análise Crítica

1. Todos os documentos possuem o mesmo padrão?

> Não. O primeiro documento possui apenas o campo `nome`; o segundo possui `nome`, `ranking` e `patrocinador`; o terceiro possui `nome`, `nick`, `ranking` e um campo extra `redeSocial`. Cada

> documento foi inserido com uma estrutura diferente, demonstrando a ausência de padrão obrigatório.

2. Isso pode gerar problemas em consultas futuras?

> Sim. Ao consultar um campo como `nick` de todos os jogadores, documentos que não possuem esse campo retornam `null` ou simplesmente não aparecem nos resultados, dificultando rankings, relatórios e integrações com APIs.

3. Cite um risco desse modelo em sistemas grandes.

> Em sistemas com grande volume de dados e múltiplos desenvolvedores, a ausência de validação pode causar **inconsistência crítica de dados**: campos com nomes grafados diferentemente (`patrocinador` vs `Patrocinador`), tipos incompatíveis (`ranking` como string em um documento e como inteiro em outro) e campos obrigatórios ausentes — comprometendo a integridade do banco e elevando o custo de manutenção.

PARTE 2

Banco Com Estrutura (BD_CE)

Tarefa 3 – Inserção Correta

Jogador inserido respeitando o schema: campos obrigatórios `nome`, `nick` e `ranking` (int ≥ 0) presentes.

```
use BD_CE

db.jogadores.insertOne({
  nome: "Lucas",
  nick: "ProPlayer",
  ranking: NumberInt(1500)
})
```

```
✓ Inserção realizada com sucesso
```

```
{ acknowledged: true, insertedId: ObjectId('...') }
```

Tarefa 4 – Inserção Inválida

Tentativa de inserir um jogador com `ranking` como string, violando o tipo definido no schema (`bsonType: "int"`):

```
db.jogadores.insertOne({
  nome: "Beatriz",
  nick: "BeatZ",
  ranking: "alto"
})
```

✗ Erro de validação de schema

```
MongoServerError: Document failed validation
Additional information: {
  failingDocumentId: ObjectId('...'),
  details: {
    operatorName: '$jsonSchema',
    schemaRulesNotSatisfied: [
      {
        operatorName: 'properties',
        propertiesNotSatisfied: [
          {
            propertyName: 'ranking',
            description: 'must be of type int'
          }
        ]
      }
    ]
  }
}
```

Tarefa 5 – Comparação Técnica

Critério	BD_SE	BD_CE
Segurança para produção	Baixa – sem validação	**Alta** – schema validado
Flexibilidade para prototipar	**Alta** – sem restrições	Média – exige conformidade
Consistência dos dados	Variável	Garantida
Cenário ideal	MVPs, provas de conceito, logs heterogêneos	APIs públicas, produção, sistemas críticos

Em qual cenário você utilizaria cada modelo?

> **BD_SE:** fases de prototipação, MVPs e contextos onde o modelo de dados ainda não está consolidado ou os documentos são altamente heterogêneos (ex: importação de eventos externos, coleta de logs).
>
> **BD_CE:** ambientes de produção onde a integridade dos dados é crítica — cadastros de usuários, transações, registros de campeonatos e qualquer módulo que alimente APIs públicas ou relatórios gerenciais.

PARTE 3

Raciocínio de Arquitetura

Situação-problema: Uma empresa de jogos está crescendo rapidamente e pretende integrar seu banco com uma API pública. O sistema será acessado por milhares de usuários simultaneamente.

→ RECOMENDAÇÃO: BD_CE

> **Integridade em escala.**
> Com milhares de inserções simultâneas, qualquer inconsistência estrutural pode propagar erros em cascata. O `$jsonSchema` garante que apenas documentos válidos sejam persistidos, eliminando a necessidade de validações duplicadas na camada de aplicação.

> **Contrato com a API pública.**
> Consumidores de uma API pública esperam respostas com estrutura previsível. Um banco sem schema pode retornar campos ausentes ou tipos incompatíveis — um `ranking` como string em vez de inteiro, por exemplo, pode quebrar clientes que esperam um número.

> **Manutenibilidade e escalabilidade.**
> Em equipes grandes, a ausência de schema dificulta o onboarding e aumenta o risco de bugs silenciosos. O schema via `$jsonSchema` serve também como documentação viva do modelo de dados.

> **Conclusão.**
> Embora o BD_SE seja útil na prototipação inicial, um sistema em produção integrado a uma API pública e com alto volume de acessos exige a confiabilidade que somente o BD_CE com schema validado pode oferecer. Estrutura não é limitação — é fundamento para escalabilidade.

BDN-Atividade-Aula03 · FATEC · Prof.ª Lucineide · 2026